

The Launcher, and Four Defects It Found by Existing

UM-NATOS-021

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-021	1.2 · 2026-08-15	**Complete, verified on hardware** — \$2.1 relaid out, \$6.5 added on chrome that cannot flicker	Internal

1. Abstract

The first thing in this project that exists for a user rather than for a test. Everything before it was verified by reading serial output; this is verified by someone touching the glass and a program starting.

It is about 250 lines. Its value turned out to be less in what it does than in what it **demand**ed — it is the first consumer to care where across the screen a finger is, the first to run the display at a rate that mattered, and the first to need a task slot. Each of those exposed a defect that had been latent for weeks:

exposed	defect	recorded in
horizontal position	touch X axis inverted since the driver was written	UM-NATOS-017 \$7.1
position validity	PENIRQ answers presence, not validity; rail samples	UM-NATOS-017 \$4.1
a task slot	nine <code>task_create</code> calls against a <code>TASK_MAX</code> of 8 — no idle task	\$5 below
display throughput	applications and renderer blocking each other on the draw lock	UM-NATOS-014 \$10

None was found by inspection. All four were found by building something that depended on them.

2. A launcher, not a window manager

Applications still render into the fixed horizontal strips `app.c` assigns by slot index. There is no focus model, no z-order, and no way for a program to own the screen and hand it back.

That boundary is deliberate rather than unfinished. Dynamic viewports need an arbitration policy for who owns which pixels, and building one before a user interface exists to demand it is designing against a guess — the same reason focus arbitration was deferred while viewports could not overlap. A launcher is the consumer that makes the question concrete; it is not yet the consumer that answers it.

2.1 Layout

```

0  +-----+-----+-----+
    | counter | squares | draw   | launcher: 3 x 4 grid
    +-----+-----+-----+ cells 80 x 52
    | paint   | blit    | ping    |
    +-----+-----+-----+
    | pong    | rogue   | 3D view | ten cells used of twelve;
    +-----+-----+-----+ the empty two cannot be selected
    | colours |          |         |
208 +-----+-----+-----+
    | status / chrome bar          | launch messages, or the open
224 +-----+-----+-----+ view's name and close button
    | application 0 | name | X | four strips, pitch 24, height 22
    +-----+-----+-----+ the right 60 px are the KERNEL's
    | application 1 | name | X |
    ...
320 +-----+-----+-----+

```

Everything left of the name belongs to the application. The name and the X do not — see §6.2.

Nothing below row 208 is drawn at boot. The strips appear when a program occupies them, and nothing starts on its own; §6.5 explains why that changed.

The 3D view is an icon rather than a permanent fixture: it becomes something you open, returning via the top-left corner.

2.2 Launch by name, never by index

`shell_launch(name)` looks the program up in the shell's table. The launcher holds no image pointers of its own.

This is not fastidiousness. An earlier launcher in this project indexed the table directly and silently started a *different* program than the one it named, because the table had been reordered — a `rogue` icon running `gfxrogue`. A name lookup cannot drift out of step with the table it reads.

2.3 The launcher costs nothing when idle

It repaints only when something changed, so an idle desktop pushes **zero** pixels — unlike the raycaster, which repaints unconditionally because every frame differs. If the screen looks frozen, that is correct: it is waiting for input.

3. Why a cursor on a touchscreen

A cursor is an indirection — you can already point at what you want. It earns its place here for two measured reasons: the panel is resistive and noisy, and an icon large enough to hit reliably by direct tap would leave room for very few.

The interaction is **hybrid, not modal**: a single tap MOVES the cursor to it, a double-tap OPENS what it is on. A confident user taps twice and it opens; an unsure one taps, sees where the cursor landed, and corrects. Neither has to be told which mode they are in.

4. Two defects of its own

4.1 Status text drawn over a control

The launch message was drawn at `DESK_H - 10`, which is inside the bottom-left cell's label. So "started" appeared over `pong`'s name — and `g_msg_sel` was never cleared, so it stayed there permanently.

It was reported as "*it selected Pong, which was a bit off*", and that reading was entirely reasonable. The interface was asserting something false about its own state, indefinitely.

Text that overlaps a control is not a cosmetic problem. The fix gives status its own strip below the grid, shrinks the cells to make room, excludes the strip from hit-testing, names what launched rather than saying "started", and expires after ~2 s — because a status line that never clears stops describing the present and becomes decoration.

4.2 The selection was read at the worst possible moment

Selection followed *every* sample while the finger was down, so the **last** sample before release decided it — and release is precisely when a resistive panel produces its worst data. Measured, one tap on the top-right icon:

```
first sample -> cell 2      (correct)
last sample  -> cell 3      (wrong)
```

The correct target was read, recorded, and discarded microseconds later.

Selection is now latched from the **first** sample of a press and not moved again until the finger lifts. Later samples describe a finger being *lifted*, which is not an instruction. Drag no longer moves the cursor; tap again to correct.

This was diagnosed by an instrument built for the theory and then nearly deleted when the theory looked wrong. The first/last cell pair is still reported, with a note saying why: removing an instrument once it has found its bug is how the bug returns unnoticed.

5. There was no idle task

`kmain` makes **nine** `task_create` calls; `TASK_MAX` was **8**. The ninth is the idle task, so it failed: `task_create()` returns `-1` when the table is full, `task_set_idle(-1)` bounds-checks and quietly does nothing, and the return value was never printed or checked.

Two real costs. `WAITI` never executed, so the CPU never slept. And with every task asleep the scheduler found nothing runnable and fell through to "resume the interrupted context" — which resumes a **sleeping** task, the exact invariant blocking exists to enforce.

The evidence had been printed and read. The `stacks` table added hours earlier listed ids 0–7 with no idle task; it was looked at and not registered. That is the failure UM-NATOS-019 is entirely about, committed while writing the report about it.

`TASK_MAX` is now 12 and `must_create()` panics rather than returning `-1`, so a full table cannot fail quietly again.

6. Icons, and a close button an application cannot reach

Added in revision 1.1.

6.1 Icons are a bitmap, not nine drawing routines

8 × 8 monochrome glyphs, one byte per row, drawn at 3× into 24 × 24. Nine bespoke drawing functions would be more code than nine constants and would make every icon a place a bug can hide; a bitmap is also editable by anyone who can count to eight, which matters more than elegance for something whose only requirement is being recognisable at 24 pixels.

Each row is drawn as one rectangle per **run** of set pixels rather than one per pixel. That is not micro-optimisation: every primitive takes the draw lock, and UM-NATOS-014 §10 established that the cost of that lock is the number of acquisitions rather than the time held.

This is still not an asset pipeline (UM-NATOS-011 §6). The glyphs are in the image because there is nowhere else to put them yet.

6.2 The close button is outside the viewport, and that is the point

Every running program gets an X, and the 3D view gets one in place of the invisible top-left corner gesture it had.

The X sits in a column the application's viewport **no longer covers**: `APP_VIEW_W` is `DISP_W` minus the chrome width, and the kernel owns those pixels.

If the close button lived inside the viewport, a misbehaving program could paint over it, draw a decoy elsewhere, or simply fill its strip and hide the way out. Outside means **the one control a user needs in order to escape a program is the one control that program cannot touch**. That is the viewport argument of UM-NATOS-016 §2 applied to a pixel the *user* owns rather than one the application does.

The touch path enforces the same boundary: close buttons see the press first, and a consumed press is not passed on, so closing a program cannot also select an icon underneath it.

6.3 A button for something you cannot name

The area below the launcher was reported as "*strange space ... why is there two x boxes in there*", and the answer was worse than the question implied.

Those were `ping` and `pong`, which `kmain` starts at boot to keep the IPC counters live. They exchange **messages**, not pixels. So two programs were running correctly, producing no visible output, and the only evidence of their existence was two buttons whose function was to destroy them. Tapping one would have been a guess about what it deleted.

Each running strip now carries its program's name beside its X.

The cost is real and is not hidden: the kernel column grew from 16 px to 60 px, so applications lost about a fifth of their strip width to a label they can neither draw nor overwrite. If a program later needs the full width, this is the decision to revisit.

6.4 The running marker, again

The mark for "this program is running" was a four-pixel dot in the corner of the icon. It is now an underline beneath the icon's label.

The dot had already failed once: it compared only the first character of the program name, so `paint`, `ping` and `pong` marked each other, and **nobody noticed by looking at the screen** — three wrong four-pixel dots read as a rendering artefact. It was found while writing §9 of this report.

A mark too small to be read wrongly is also too small to be read at all. The underline spans the cell.

6.5 Chrome over a repainting view cannot be ordered into working

Added in revision 1.2.

Two things held permanent screen space without earning it: a band of application strips, and an animated colour strip along the bottom. Asked what that area was, the answer was that the strips were empty and the X boxes in them belonged to `ping` and `pong` — started automatically by `kmain`, exchanging messages rather than drawing. A boot that hands the user two controls whose only purpose is to destroy programs they did not start.

Nothing starts on its own now, and the colour artwork became a menu entry: open it and it takes the region with a close button, exactly as the 3D view does. Both are expressed as a **mode** rather than a pile of booleans, so "who owns the region" has one answer.

That freed 154 rows, and the grid grew to 3×4 . The grid is sized by the **screen** rather than by the current program list, so adding a program does not mean re-deciding the layout: two of the twelve cells are empty, draw nothing, and cannot be selected.

The flicker, and why ordering could not fix it

The close button for a full-region view was first drawn *after* the view, each frame. It strobed badly.

The raycaster repaints **every pixel every frame**, so anything drawn over it is visible only in the gap between one repaint and the next — at 16 fps with a 29 ms blit, that gap is most of the frame missing. Drawing the button later in the sequence does not help, because "later" only lasts until the next frame begins.

The fix is ownership, not ordering. The region now ends at row 208 and a 16-row bar below it belongs to the kernel; **no view draws into those rows**, so the button is painted over pixels nothing else touched. `RAY_VIEW_H` shrank from 224 to 208 accordingly, which also returned 7,680 bytes of framebuffer.

It read as "the 3D view is broken". The frames were correct throughout — a strobing element in the corner is enough to make a working view look faulty, which is worth remembering before debugging the renderer.

7. Verification

```
tap left icon   raw_x=3408 -> x=0,   cell 0
tap right icon  raw_x=920  -> x=196, cell 2
seven consecutive taps, z 1662..2261, no rail samples
x spanning 0..187 across all three columns
first and last sample of each press agree
```

All three columns selectable, double-tap opens the named program, and the status strip reports which. Confirmed on hardware by the user.

8. Metrics

Quantity	Value
Grid	3 × 3, cells 80 × 51 px
Status strip	14 px, message expires ~2 s
Double-tap window	60 ticks (~600 ms)
Minimum press	2 ticks, rejects contact chatter
SPI cost when idle	0 bytes
Grid	3 × 4, cells 80 × 52; ten cells used of twelve
Icon glyphs	8 × 8, drawn at 3×
Chrome bar	16 rows, owned by the kernel, no view draws there
Kernel-owned column	60 px of 240 (name + close button)
Application viewport width	180 px, was 240
Defects exposed in other subsystems	4
Defects of its own	2

9. What this does not establish

- **No focus, no windows, no z-order.** §2. Applications occupy fixed strips and cannot be brought forward.
- **No way to stop a program from the launcher.** `kill` exists only in the shell; the icon grid can start and cannot stop.
- **The running marker is per-name, not per-instance.** Two instances of the same program would share one underline. §6.4.
- **Faulted programs cannot be dismissed.** A program that faults keeps its slot and shows no X, because it is not running — it lingers until `kill` from the shell. The new chrome makes that visible without addressing it.
- **The chrome column is fixed width.** 60 px regardless of how long the name is, and names are truncated to fit rather than scaled or scrolled.
- **Two grid cells are empty.** They draw nothing and reject touches, which is deliberate and looks like a bug until you know that.
- **`ping` and `pong` no longer start at boot,** so the IPC counters read zero until someone launches them — ping first, since it addresses application 1 and slots go lowest-free-first. Continuous messaging evidence is now opt-in.
- **Double-tap timing is unmeasured.** 600 ms and the 2-tick minimum press were reasoned, not derived from anyone's actual tapping. The counters exist to settle them and nobody has.
- **The icons are guesses at 24 pixels.** 8 × 8 glyphs scaled 3×, drawn by hand and judged by one person. `blit` and `rogue` are the two most likely to read as noise. Nothing tests whether any of them is recognisable.

- **One user.** Every judgement about whether the interaction feels right came from a single person on a single panel.

10. References

- UM-NATOS-017 §4.1, §7.1 — the touch defects this exposed
- UM-NATOS-014 §10 — the lock contention this exposed
- UM-NATOS-019 — verifying that a mechanism exists is not verifying it works
- UM-NATOS-016 §2 — the fixed viewport model the launcher does not change
- `kernel/desktop.c` — grid, cursor, press latching, status strip
- `kernel/shell.c` — `shell_launch()` , the single lookup path